

```

1      *****CH_01_JAVA_QB_Solution_2024*****
2
3      /*
4      Q_1:Write a java program to explain runtime polymorphism using interfaces
5      Q:2 write a program that implements dynamic method dispatch.
6      Q:3 Create class Fruit with method void display(). Create subclasses Apple and Banana
       that overrides display().
7      In main method kindly show the concept of Dynamic method dispatch by creating reference
       of Fruit and object of Apple and Banana respectively and call display().
8      */
9      class Fruit
10     {
11         void display()
12         {
13             System.out.println("Method of class Fruit");
14         }
15     }
16     class Apple extends Fruit
17     {
18         void display()
19         {
20             System.out.println("Method of class Apple");
21         }
22     }
23     class Banana extends Fruit
24     {
25         void display()
26         {
27             System.out.println("Method of class Banana");
28         }
29     }
30     class runFruit
31     {
32         public static void main(String[] args)
33         {
34             Fruit oba = new Apple();
35             oba . display(); //Method of class Apple
36
37             Fruit obb = new Banana();
38             obb . display(); //Method of class Banana
39         }
40     }
41
42     /*QB:12 Declare a class called book having author_name as private data
43     member. Extend book class to have two sub classes called
44     book_publication & paper_publication. Each of these classes have
45     private member called title. Write a complete program to show usage
46     of dynamic method dispatch (dynamic polymorphism) to display book or
47     paper publications of given author. Use command line arguments for
48     inputting data.*/
49
50     class Book
51     {
52         private String author_Name;
53

```

```
54     public Book(String author_Name)
55     {
56         this.author_Name = author_Name;
57     }
58     String show()
59     {
60         return author_Name;
61     }
62     public void display()
63     {
64         System.out.println("Author Name::"+author_Name);
65         System.out.println();
66     }
67 }
68 class Book_publication extends Book
69 {
70     private String title;
71
72     public Book_publication(String author_Name, String title)
73     {
74         super(author_Name);
75         this.title = title;
76     }
77     public void display()
78     {
79         System.out.println("Book Publication");
80         System.out.println("Title::"+title);
81         super.display();
82     }
83 }
84 class Paper_publication extends Book
85 {
86     private String title;
87
88     public Paper_publication(String author_Name, String title)
89     {
90         super(author_Name);
91         this.title = title;
92     }
93     public void display()
94     {
95         System.out.println("Paper Publication");
96         System.out.println("Title::"+title);
97         super.display();
98     }
99 }
100 class Main
101 {
102     public static void main(String[] args) {
103         String BookAuthor=args[0];
104         String BookTitle=args[1];
105         String PaperAuthor=args[2];
106         String PaperTitle=args[3];
107
108         Book B1=new Book_publication(BookAuthor,BookTitle);
```

```

109         Book B2=new Paper_publication(PaperAuthor, PaperTitle);
110         B1.display();
111         B2.display();
112
113     }
114 }
115 /*PS F:\JAVA QB\Java-II\CHI Prog> java Main Hridank HarryPorter Ankur
116 MachineDesign
117
118 Book Publication
119 Title::HarryPorter
120 Author Name::Hridank
121
122 Paper Publication
123 Title::MachineDesign
124 Author Name::Ankur */
125
126
127 QB:15 /*Write a abstract class named Person and its two subclasses
128 named student and Employee. A person has a name, address, phone number
129 and email address. A student has enrollment course.An Employee has an
130 office, salary, and designation. Define constructors and methods for
131 input and display for both classes. Write a main java program to give
132 demonstration of all. */
133
134 import java.util.Scanner;
135
136 abstract class Person {
137     String name;
138     String address;
139     String phoneNumber;
140     String emailAddress;
141
142     public Person(String name, String address, String phoneNumber, String
143 emailAddress) {
144         this.name = name;
145         this.address = address;
146         this.phoneNumber = phoneNumber;
147         this.emailAddress = emailAddress;
148     }
149     abstract void display();
150 }
151
152 class Student extends Person {
153     String enrollmentCourse;
154
155     public Student(String name, String address, String phoneNumber, String
156 emailAddress, String enrollmentCourse) {
157         super(name, address, phoneNumber, emailAddress);
158         this.enrollmentCourse = enrollmentCourse;
159     }
160
161     void display() {
162         System.out.println(":::::::::Student Details:::::::::");
163         System.out.println(":::::::::::::::::::::::::::::::::");

```

```

164         System.out.println("Name:" + super.name);
165         System.out.println("Address:" + super.address);
166         System.out.println("Phone Number:" + super.phoneNumber);
167         System.out.println("Email Address:" + super.emailAddress);
168         System.out.println("Enroll Coure:" + enrollmentCourse);
169         System.out.println("::::::::::::::::::::::::::::::::");
170         System.out.println();
171     }
172 }
173
174 class Employee extends Person {
175     String office;
176     double salary;
177     String designation;
178
179     public Employee(String name, String address, String phoneNumber,
180 String emailAddress, String office, double salary, String designation)
181 {
182         super(name, address, phoneNumber, emailAddress);
183         this.office = office;
184         this.salary = salary;
185         this.designation = designation;
186     }
187
188     void display() {
189         System.out.println("::::::::Employee Details::::::::");
190         System.out.println("::::::::::::::::::::::::::::::::");
191         System.out.println("Name:" + super.name);
192         System.out.println("Address:" + super.address);
193         System.out.println("Phone Number:" + super.phoneNumber);
194         System.out.println("Email Address:" + super.emailAddress);
195         System.out.println("Office:" + office);
196         System.out.println("Salary:" + salary);
197         System.out.println("Designation:" + designation);
198         System.out.println("::::::::::::::::::::::::::::::::");
199         System.out.println();
200     }
201 }
202
203 class Main1 {
204     public static void main(String[] args) {
205         Scanner sc = new Scanner(System.in);
206         System.out.println(":::::Enter Details of Student:::::");
207         System.out.print("Enter Name:");
208         String name = sc.nextLine();
209         System.out.print("Enter Address:");
210         String address = sc.nextLine();
211         System.out.print("Enter Phone-Number:");
212         String phoneNumber = sc.nextLine();
213         System.out.print("Enter Email:");
214         String emailAddress = sc.nextLine();
215         System.out.print("Enter enrollmentCourse:");
216         String enrollmentCourse = sc.nextLine();
217         System.out.println();
218

```

```

219         Student s = new Student(name, address, phoneNumber, emailAddress,
220 enrollmentCourse);
221
222         System.out.println(":::::Enter Details of Employee:::::");
223         System.out.print("Enter Name:");
224         String name1 = sc.nextLine();
225         System.out.print("Enter Address:");
226         String address1 = sc.nextLine();
227         System.out.print("Enter Phone-Number:");
228         String phoneNumber1 = sc.nextLine();
229         System.out.print("Enter Email:");
230         String emailAddress1 = sc.nextLine();
231         System.out.print("Enter Office:");
232         String office = sc.nextLine();
233         System.out.print("Enter Salary:");
234         double salary = sc.nextDouble();
235         System.out.print("Enter Designation:");
236         sc.nextLine();
237         String designation = sc.nextLine();
238         System.out.println();
239
240         Employee e = new Employee(name1, address1, phoneNumber1,
241 emailAddress1, office, salary, designation);
242         s.display();
243         e.display();
244     }
245 }
246 /*
247 ::::::::::Student Details::::::::::
248 ::::::::::::::::::::::::::::::::::::
249 Name:AP
250 Address:AMD
251 Phone Number:9898915690
252 Email Address:ap.pp@gmail.com
253 Enroll Coure:ME
254 ::::::::::::::::::::::::::::::::::::
255
256 ::::::::::Employee Details::::::::::
257 ::::::::::::::::::::::::::::::::::::::
258 Name:VP
259 Address:Gnagar
260 Phone Number:8787965421
261 Email Address:VP.PP@gmail.com
262 Office:TCS
263 Salary:75000.0
264 Designation:Manager
265 ::::::::::::::::::::::::::::::::::::::
266 */
267
268
269
270 QB:16:-/*Describe abstract class called Shape which has three sub
271 classes say Triangle, Rectangle,Circle. Define one method area() in
272 the abstract class and override this area() in these three subclasses
273 to calculate for specific object i.e. area() of Triangle subclass

```

```
274  should calculate area of triangle etc. Same for Rectangle and Circle.
275  */
276
277  abstract class Shape
278  {
279      public abstract double area();
280  }
281
282  class Triangle extends Shape
283  {
284      private double base;
285      private double height;
286
287      public Triangle(double base, double height)
288      {
289          this.base = base;
290          this.height = height;
291      }
292
293      @Override
294      public double area()
295      {
296          return 0.5 * base * height;
297      }
298  }
299
300  class Rectangle extends Shape
301  {
302      private double length;
303      private double width;
304
305      public Rectangle(double length, double width)
306      {
307          this.length = length;
308          this.width = width;
309      }
310
311      @Override
312      public double area()
313      {
314          return length * width;
315      }
316  }
317
318  class Circle extends Shape
319  {
320      private double radius;
321
322      public Circle(double radius)
323      {
324          this.radius = radius;
325      }
326
327      @Override
328      public double area()
```

```

329     {
330         return Math.PI * radius * radius;
331     }
332 }
333 class Main2
334 {
335     public static void main(String[] args) {
336         // Create objects of Triangle, Rectangle, and Circle
337         Triangle triangle = new Triangle(5, 8);
338         Rectangle rectangle = new Rectangle(4, 6);
339         Circle circle = new Circle(3);
340
341         // Call area() method for each object
342         double triangleArea = triangle.area();
343         double rectangleArea = rectangle.area();
344         double circleArea = circle.area();
345
346         // Display the areas
347         System.out.println("Area of Triangle: " + triangleArea);
348         System.out.println("Area of Rectangle: " + rectangleArea);
349         System.out.println("Area of Circle: " + circleArea);
350     }
351 }
352 /*
353 Area of Triangle: 20.0
354 Area of Rectangle: 24.0
355 Area of Circle: 28.274333882308138 */
356
357 QB:17:-/*The abstract Vegetable class has three subclasses named
358 Potato, Brinjal and Tomato. Write an application that demonstrates
359 how to establish this class hierarchy. Declare one instance variable
360 of type String that indicates the color of a vegetable. Create and
361 display instances of these objects. Override the toString() method of
362 Object to return a string with the name of the vegetable and its
363 color. */
364
365 abstract class Vegetable
366 {
367     String color;
368     @Override
369     public String toString()
370     {
371         return "";
372     }
373 }
374 class Potato extends Vegetable
375 {
376     String color;
377
378     public Potato(String color) {
379         this.color = color;
380     }
381
382     @Override
383     public String toString() {

```

```

384         return "Potato [color=" + color + "];"
385     }
386
387 }
388
389 class Brinjal extends Vegetable
390 {
391     String color;
392
393     public Brinjal(String color) {
394         this.color = color;
395     }
396
397     @Override
398     public String toString() {
399         return "Brinjal [color=" + color + "];"
400     }
401 }
402
403 class Tomato extends Vegetable
404 {
405     String color;
406
407     public Tomato(String color) {
408         this.color = color;
409     }
410
411     @Override
412     public String toString() {
413         return "Tomato [color=" + color + "];"
414     }
415 }
416
417
418 class Main4 {
419     public static void main(String[] args) {
420         // Create instances of Potato, Brinjal, and Tomato
421         Potato potato = new Potato("Brown");
422         Brinjal brinjal = new Brinjal("Purple");
423         Tomato tomato = new Tomato("Red");
424
425         // Display the objects using overridden toString() method
426         System.out.println("Potato: " + potato);
427         System.out.println("Brinjal: " + brinjal);
428         System.out.println("Tomato: " + tomato);
429     }
430 }
431 /*
432 Potato: Potato [color=Brown]
433 Brinjal: Brinjal [color=Purple]
434 Tomato: Tomato [color=Red] */
435
436 QB-21:- /* Java program to implement interface with hierarchical inheritance*/
437 interface I1
438 {

```

```

439     public void m1();
440 }
441
442 class I2 implements I1
443 {
444     public void m1(){
445         System.out.println("bye");
446     }
447 }
448
449 class I3 implements I1
450 {
451     public void m1(){
452         System.out.println("Hi");
453     }
454 }
455 class Main{
456     public static void main(String[] args) {
457         I3 a = new I3();
458         a.m1();
459         I2 b = new I2();
460         b.m1();
461     }
462 }
463
464
465 QB-25:-/*Write a java program to implement the multiple inheritance
466 concepts for calculating area of circle and square*/
467
468 //using hierarchical inheritance ---->
469 // Interface for shapes that have area
470
471 interface AreaCalculatable
472 {
473     double calculateArea();
474 }
475
476 // Circle class implementing AreaCalculatable interface
477
478 class Circle implements AreaCalculatable
479 {
480     private double radius;
481
482     public Circle(double radius)
483     {
484         this.radius = radius;
485     }
486     public double calculateArea()
487     {
488         return Math.PI * radius * radius;
489     }
490 }
491
492 // Square class implementing AreaCalculatable interface
493

```

```
494 class Square implements AreaCalculatable
495 {
496     private double side;
497
498     public Square(double side)
499     {
500         this.side = side;
501     }
502
503     public double calculateArea()
504     {
505         return side * side;
506     }
507 }
508
509 class AreaCalculator
510 {
511     public static void main(String[] args)
512     {
513         // Create instances of Circle and Square
514         Circle circle = new Circle(5);
515         Square square = new Square(4);
516
517         // Calculate and print areas
518         System.out.println("Area of Circle: " + circle.calculateArea());
519         System.out.println("Area of Square: " + square.calculateArea());
520     }
521 }
522
523 //using multiple inheritance --->
524 interface Circle {
525     double calculateArea();
526 }
527
528 interface Square {
529     double calculateArea();
530 }
531
532 class CircleImpl implements Circle {
533     private double radius;
534
535     public CircleImpl(double radius) {
536         this.radius = radius;
537     }
538
539     @Override
540     public double calculateArea() {
541         return Math.PI * radius * radius;
542     }
543 }
544
545 class SquareImpl implements Square {
546     private double side;
547
548     public SquareImpl(double side) {
```

```

549         this.side = side;
550     }
551
552     @Override
553     public double calculateArea() {
554         return side * side;
555     }
556 }
557
558 class CircleAndSquare implements Circle, Square {
559     private Circle circle;
560     private Square square;
561
562     public CircleAndSquare(double radius, double side) {
563         this.circle = new CircleImpl(radius);
564         this.square = new SquareImpl(side);
565     }
566
567     @Override
568     public double calculateArea() {
569         // You can choose either circle's or square's area calculation
570         // Here, I'm choosing to return the area of the circle
571         return circle.calculateArea();
572     }
573 }
574
575 public class Main {
576     public static void main(String[] args) {
577         double circleRadius = 5.0;
578         double squareSide = 4.0;
579
580         CircleAndSquare combined = new CircleAndSquare(circleRadius, squareSide);
581
582         System.out.println("Area of Circle: " + combined.calculateArea());
583         // If you want to calculate the area of square, you can do it like this:
584         // System.out.println("Area of Square: " + combined.calculateSquareArea());
585     }
586 }
587
588
589 QB-26, 28, 29:-
590 Write a program that illustrates interface inheritance. Interface P is
591 extended by P1 And P2. Interface P12 inherits from both P1 and P2. Each
592 interface declares one constant and one method. Class Q implements
593 P12. Instantiate Q and invokes each of its methods. Each method displays
594 one of the constants
595
596 // Define the interface P
597 interface P
598 {
599     int CONSTANT_P = 10;
600     void methodP();
601 }
602
603 // Define the interface P1 extending P

```

```
604 interface P1 extends P
605 {
606     int CONSTANT_P1 = 20;
607     void methodP1();
608 }
609
610 // Define the interface P2 extending P
611 interface P2 extends P
612 {
613     int CONSTANT_P2 = 30;
614     void methodP2();
615 }
616
617 // Define the interface P12 inheriting from both P1 and P2
618 interface P12 extends P1, P2
619 {
620     int CONSTANT_P12 = 40;
621     void methodP12();
622 }
623
624 // Define class Q implementing P12
625 class Q implements P12
626 {
627     public void methodP()
628     {
629         System.out.println("Method P invoked with constant: " + CONSTANT_P);
630     }
631     public void methodP1()
632     {
633         System.out.println("Method P invoked with constant: " + CONSTANT_P1);
634     }
635     public void methodP2()
636     {
637         System.out.println("Method P invoked with constant: " + CONSTANT_P2);
638     }
639     public void methodP12()
640     {
641         System.out.println("Method P12 invoked with constant:"+CONSTANT_P12);
642     }
643 }
644
645 class QB_26
646 {
647     public static void main(String[] args)
648     {
649         // Instantiate Q
650         Q q = new Q();
651         // Invoke methods
652         q.methodP();
653         q.methodP1();
654         q.methodP2();
655         q.methodP12();
656     }
657 }
658
```

```

659
660 QB-27: -/*Create interface Univerity with method takeExam(). Create
661 Interface State with method getName(). Create interface StateUniversity
662 that extends University and State. Create print() inside
663 StateUniversity. Create class TestState that implements StateUniversity
664 interface. and Create a class withmain method that calls all methods of
665 TestState class */
666
667 // Interface University
668
669 interface University
670 {
671     void takeExam();
672 }
673
674 // Interface State
675
676 interface State
677 {
678     String getName();
679 }
680
681 // Interface StateUniversity extending both University and State
682
683 interface StateUniversity extends University, State
684 {
685     void print();
686 }
687
688 // Class TestState implementing StateUniversity interface
689
690 class TestState implements StateUniversity
691 {
692
693     public void takeExam()
694     {
695         System.out.println("Taking exam...");
696     }
697     public String getName()
698     {
699         return "State University";
700     }
701     public void print()
702     {
703         System.out.println("State University's name: " + getName());
704     }
705 }
706 class QB_27
707 {
708     public static void main(String[] args)
709     {
710         TestState testState = new TestState();
711
712         // Calling methods of TestState class
713         testState.takeExam();

```

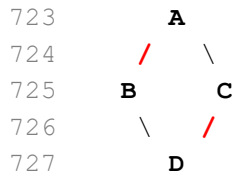
```
714         testState.print();
715     }
716 }
```

718 QB-28 : same as 26

719 QB-29 : same as 26

720

721 QB-32:- Write a program to demonstrate the multipath inheritance for the classes having relations as shown in figure A-> (B, C) ->D.



728 Ans:

729 In Java, classes do not support multiple inheritance directly (2 parent -> 1 child).
730 However, you can achieve a form of multipath inheritance using interfaces.

731

```
732 // Define interface A
733 interface A {
734     void methodA();
735 }
736
737 // Define interface B extending A
738 interface B extends A {
739     void methodB();
740 }
741
742 // Define interface C extending A
743 interface C extends A {
744     void methodC();
745 }
746
747 // Define interface D extending both B and C
748 interface D extends B, C {
749     void methodD();
750 }
751
752 // Concrete class implementing interface D
753 class MyClass implements D {
754     @Override
755     public void methodA() {
756         System.out.println("Inside methodA");
757     }
758
759     @Override
760     public void methodB() {
761         System.out.println("Inside methodB");
762     }
763
764     @Override
765     public void methodC() {
766         System.out.println("Inside methodC");
767     }
768 }
```

```

768
769     @Override
770     public void methodD() {
771         System.out.println("Inside methodD");
772     }
773 }
774
775 public class Main {
776     public static void main(String[] args) {
777         MyClass obj = new MyClass();
778         obj.methodA(); // Method from interface A
779         obj.methodB(); // Method from interface B
780         obj.methodC(); // Method from interface C
781         obj.methodD(); // Method from interface D
782     }
783 }
784

```

785 **QB-33:-** Explain single level and multiple inheritances in java. Write a program to
786 demonstrate
787 combination of both types of inheritance as shown in figure 1. i.e. hybrid inheritance .

```

788  A      B
789  \    /
790   C
791   |
792   D
793
794 // Define interface A
795 interface A {
796     void methodA();
797 }
798
799 // Define interface B
800 interface B {
801     void methodB();
802 }
803
804 // Define interface C extending A and B
805 interface C extends A, B {
806     void methodC();
807 }
808
809 // Define class D implementing interface C
810 class D implements C {
811     @Override
812     public void methodA() {
813         System.out.println("Inside methodA");
814     }
815
816     @Override
817     public void methodB() {
818         System.out.println("Inside methodB");
819     }
820
821     @Override
822     public void methodC() {

```

```

822         System.out.println("Inside methodC");
823     }
824 }
825
826 public class Main {
827     public static void main(String[] args) {
828         D obj = new D();
829         obj.methodA(); // Method from interface A
830         obj.methodB(); // Method from interface B
831         obj.methodC(); // Method from interface C
832     }
833 }
834

```

835 **QB-34:-**The Transport interface declares a deliver() method. The abstract
836 class Animal is the superclass of the Tiger, Camel, Deer and Donkey
837 classes. The Transport interface is implemented by the Camel and Donkey
838 classes. Write a test program that initialize an array of four Animal
839 objects. If the object implements the Transport interface, the deliver()
840 method is invoked.

```

841 // Transport interface
842 interface Transport
843 {
844     void deliver();
845 }
846
847 // Abstract class Animal
848 abstract class Animal
849 {
850     // Abstract method
851     abstract void sound();
852 }
853
854 // Tiger class extending Animal
855 class Tiger extends Animal
856 {
857     void sound()
858     {
859         System.out.println("Tiger roars!");
860     }
861 }
862
863 // Camel class extending Animal and implementing Transport
864 class Camel extends Animal implements Transport
865 {
866     void sound()
867     {
868         System.out.println("Camel grunts!");
869     }
870
871     public void deliver()
872     {
873         System.out.println("Camel delivers goods.");
874     }
875 }
876

```

```

877
878 // Deer class extending Animal
879 class Deer extends Animal
880 {
881     void sound()
882     {
883         System.out.println("Deer barks!");
884     }
885 }
886
887 // Donkey class extending Animal and implementing Transport
888 class Donkey extends Animal implements Transport
889 {
890     void sound()
891     {
892         System.out.println("Donkey brays!");
893     }
894
895     public void deliver()
896     {
897         System.out.println("Donkey carries loads.");
898     }
899 }
900
901 class QB_34
902 {
903     public static void main(String[] args)
904     {
905         // Initialize an array of Animal objects
906         Animal[] animals = {new Tiger(), new Camel(), new Deer(), new Donkey()};
907
908         // Iterate over the array and check if the object implements
909         Transport interface
910         // If it does, invoke the deliver() method
911         for (Animal animal : animals)
912         {
913             if (animal instanceof Transport)
914             {
915                 ((Transport) animal).deliver();
916             }
917         }
918     }
919 }
920

```

QB-35:-

Create an **interface** named Polygon. It has a **default** method `getSides()` and an **abstract** method `getArea()`. create two classes Rectangle and Square that implement Polygon. The Rectangle **class** provides the implementation of the `getArea()` method and overrides the `getSides()` method. the Square **class** only provides the implementation of the `getArea()` method. while calling the `getSides()` method using the Rectangle object, the overridden method is called. However, in the **case** of the Square object, the **default** method is called.

```

930
931 // Define the interface Polygon

```

```
932 interface Polygon
933 {
934     // Abstract method
935     double getArea();
936
937     // Default method
938     default int getSides()
939     {
940         return 4; // Default number of sides for a polygon
941     }
942 }
943
944 // Rectangle class implementing Polygon
945 class Rectangle implements Polygon
946 {
947     double length;
948     double width;
949
950     // Constructor
951     Rectangle(double length, double width)
952     {
953         this.length = length;
954         this.width = width;
955     }
956
957     // Override getArea method
958     public double getArea()
959     {
960         return length * width;
961     }
962
963     // Override getSides method
964
965     public int getSides()
966     {
967         return Polygon.super.getSides(); // Rectangle has 4 sides
968     }
969 }
970
971 // Square class implementing Polygon
972 class Square implements Polygon
973 {
974     double side;
975
976     // Constructor
977     Square(double side)
978     {
979         this.side = side;
980     }
981
982     // Override getArea method
983
984     public double getArea()
985     {
986         System.out.println("Number of sides of Square : " + Polygon.super.getSides());
```

```

987         return side * side;
988     }
989 }
990 class QB_35
991 {
992     public static void main(String[] args)
993     {
994
995         Rectangle rectangle = new Rectangle(5, 4);
996
997         System.out.println("Number of sides of Rectangle: " + rectangle.getSides());
998         System.out.println("Area of Rectangle: " + rectangle.getArea());
999
1000
1001         Square square = new Square(5);
1002
1003         System.out.println("Area of square : " + square.getArea());
1004
1005     }
1006 }
1007
1008 QB 42:- Write a program to demonstrate the multipath inheritance for the classes having
relations as
shown in figure:
1009
1010 A    B  C
1011 \  /  /
1012  D  /
1013   \ /
1014   E
1015
1016 Ans: In Java, classes do not support multiple inheritance directly.
1017 However, you can achieve a form of multipath inheritance using interfaces.
1018 // Define interface A
1019 interface A {
1020     void methodA();
1021 }
1022
1023 // Define interface B
1024 interface B {
1025     void methodB();
1026 }
1027
1028 // Define interface C
1029 interface C {
1030     void methodC();
1031 }
1032
1033 // Define class D implementing A and B
1034 class D implements A, B {
1035     @Override
1036     public void methodA() {
1037         System.out.println("Inside methodA");
1038     }
1039
1040     @Override

```

```

1041     public void methodB() {
1042         System.out.println("Inside methodB");
1043     }
1044 }
1045
1046 // Define class E inheriting from D and implementing C
1047 class E extends D implements C {
1048     @Override
1049     public void methodC() {
1050         System.out.println("Inside methodC");
1051     }
1052 }
1053
1054 public class Main {
1055     public static void main(String[] args) {
1056         E obj = new E();
1057         obj.methodA(); // Method from interface A
1058         obj.methodB(); // Method from interface B
1059         obj.methodC(); // Method from interface C
1060     }
1061 }

```

QB-43:-

- Q1) Create an **abstract class** pen with methods write () and refill () as **abstract methods**
- Q2) Use the pen **class** from Q1 to create a concrete **class** fountain pen with additional method change Nib ()
- Q3) Create a **class** monkey with jump () and bite() methods Create a **class** human Which inherits **this** monkey **class** and **implements** basicanimal **interface** with eat() and sleep methods
- Q4) Create a **class** telephone with (), lift () and disconnected () methods as **abstract methods** create another **class** smart telephone and demonstrate polymorphism
- Q5) Demonstrate polymorphism using using monkey **class** from Q3
- Q6) Create an **interface** TVremote and use it to inherit another **interface** smart TVremote
- Q7) Create a **class** TV which **implements** TVremote **interface** from Q6

```

1079 // Q1) Create an abstract class Pen
1080 abstract class Pen
1081 {
1082     // Abstract methods
1083     abstract void write();
1084     abstract void refill();
1085 }
1086
1087 // Q2) Create a concrete class FountainPen extending Pen
1088 class FountainPen extends Pen
1089 {
1090     // Implementation of abstract methods
1091
1092     void write()
1093     {
1094         System.out.println("Writing with Fountain Pen");
1095     }

```

```
1096     void refill()
1097     {
1098         System.out.println("Refilling Fountain Pen");
1099     }
1100     // Additional method
1101     void changeNib()
1102     {
1103         System.out.println("Changing nib of Fountain Pen");
1104     }
1105 }
1106
1107 // Q3) Create a class Monkey with jump() and bite() methods
1108 class Monkey
1109 {
1110     void jump()
1111     {
1112         System.out.println("Monkey jumps");
1113     }
1114     void bite()
1115     {
1116         System.out.println("Monkey bites");
1117     }
1118 }
1119
1120 // Create an interface BasicAnimal
1121 interface BasicAnimal
1122 {
1123     void eat();
1124     void sleep();
1125 }
1126
1127 // Q3) Create a class Human which inherits Monkey and implements BasicAnimal interface
1128 class Human extends Monkey implements BasicAnimal
1129 {
1130     // Implementation of BasicAnimal interface methods
1131
1132     public void eat()
1133     {
1134         System.out.println("Human eats");
1135     }
1136     public void sleep()
1137     {
1138         System.out.println("Human sleeps");
1139     }
1140 }
1141
1142 // Q4) Create an abstract class Telephone
1143 abstract class Telephone
1144 {
1145     // Abstract methods
1146     abstract void dial();
1147     abstract void lift();
1148     abstract void disconnect();
1149 }
1150
```

```

1151 // Q4) Create another class SmartTelephone inheriting from Telephone
1152 class SmartTelephone extends Telephone
1153 {
1154     // Implementation of abstract methods
1155     void dial()
1156     {
1157         System.out.println("Dialing a number on Smart Telephone");
1158     }
1159     void lift()
1160     {
1161         System.out.println("Lifting the receiver of Smart Telephone");
1162     }
1163     void disconnect()
1164     {
1165         System.out.println("Disconnecting call on Smart Telephone");
1166     }
1167
1168     // Additional method
1169     void browseInternet()
1170     {
1171         System.out.println("Browsing the internet on Smart Telephone");
1172     }
1173 }
1174
1175 // Q5) Demonstrate polymorphism using Monkey class
1176 class PolymorphismDemo
1177 {
1178     void performActions(Monkey monkey)
1179     {
1180         monkey.jump(); // Polymorphic method call
1181         monkey.bite(); // Polymorphic method call
1182     }
1183 }
1184
1185 // Q6) Create an interface TVRemote
1186 interface TVRemote
1187 {
1188     void turnOn();
1189     void turnOff();
1190     void changeChannel(int channel);
1191 }
1192
1193 // Create another interface SmartTVRemote inheriting TVRemote
1194 interface SmartTVRemote extends TVRemote
1195 {
1196     void browseInternet();
1197 }
1198
1199 // Q7) Create a class TV which implements TVRemote interface
1200 class TV implements TVRemote
1201 {
1202     // Implementation of TVRemote interface methods
1203     public void turnOn()
1204     {
1205         System.out.println("TV is turned on");

```

```

1206     }
1207     public void turnOff()
1208     {
1209         System.out.println("TV is turned off");
1210     }
1211     public void changeChannel(int channel)
1212     {
1213         System.out.println("Changing channel to " + channel);
1214     }
1215 }
1216 class QB_43
1217 {
1218     public static void main(String[] args)
1219     {
1220         // Q2) Create an instance of FountainPen
1221         FountainPen fountainPen = new FountainPen();
1222         fountainPen.write();
1223         fountainPen.refill();
1224         fountainPen.changeNib();
1225
1226         // Q3) Create an instance of Human and demonstrate inherited methods
1227         Human human = new Human();
1228         human.jump();
1229         human.bite();
1230         human.eat();
1231         human.sleep();
1232
1233         // Q5) Demonstrate polymorphism using Monkey class
1234         PolymorphismDemo demo = new PolymorphismDemo();
1235         Monkey monkey = new Monkey();
1236         demo.performActions(monkey);
1237
1238         // Q7) Create an instance of TV and demonstrate TVRemote interface methods
1239         TV tv = new TV();
1240         tv.turnOn();
1241         tv.changeChannel(5);
1242         tv.turnOff();
1243
1244         // Q4) Create an instance of SmartTelephone and demonstrate polymorphism
1245         Telephone telephone = new SmartTelephone();
1246         telephone.dial();
1247         telephone.lift();
1248         telephone.disconnect();
1249
1250         // We can't directly call browseInternet method as it's not available in
1251         // Telephone class.
1252         // However, we can create a SmartTelephone reference to access browseInternet
1253         // method
1254         SmartTelephone smartTelephone = (SmartTelephone) telephone;
1255         smartTelephone.browseInternet();
1256     }
1257 }
1258 /*
1259 * Q_44: Write a program that demonstrates the instanceof operator. Declare interfaces I1

```

```
and I2.
1259 Interface I3 extends both of these interfaces.Also declare interface I4.
1260 class X implements I3.Class W extends X and implements I4.create an object of class W.
1261 Use the instanceof operator to test if that object implements each of the interfaces
and is of type X.
1262 */
1263 interface I1
1264 {
1265 }
1266
1267 interface I2
1268 {
1269 }
1270
1271 interface I3 extends I1,I2
1272 {
1273 }
1274
1275 interface I4
1276 {
1277 }
1278
1279 class X implements I3
1280 {
1281 }
1282
1283 class W extends X implements I4
1284 {
1285 }
1286
1287 class runQ_39
1288 {
1289     public static void main( String args[] )
1290     {
1291         W wObj = new W();
1292
1293         if ( wObj instanceof I1 )
1294             System.out.println(" W implements I1 ");
1295         if ( wObj instanceof I2 )
1296             System.out.println(" W implements I2 ");
1297         if ( wObj instanceof I3 )
1298             System.out.println(" W implements I3 ");
1299         if ( wObj instanceof I4 )
1300             System.out.println(" W implements I4 ");
1301         if ( wObj instanceof X )
1302             System.out.println(" W extends X ");
1303     }
1304 }
1305
```